# LAB NO. 8

# POLYMORPHISM

# METHOD OVERRIDING (VIRTUAL METHODS) IN PYTHON

**Lab Objectives**

Following are the lab objectives:

1. Understand method overriding
2. Implement method overriding

| **Student ID** | **9864** | **Student Name** | **Ali Raza** |
|---|---|---|---|
| | **Marks** | **Obtained Marks** | **Comments** |
| **Task 1** | **20** | | |
| **Task 2** | **20** | | |
| **Task 3** | **20** | | |
| **Total Marks** | **60** | | |

**Lab Instructor**

## Lab Objectives and CLOs Mapping

| Lab Objectives | CLOs | | |
|---|---|---|---|
| | a | b | C |
| 1 | | | |
| 2 | | | |

## Instructions

- This is individual Lab work/task.
- Complete this lab work within lab timing.
- Discussion with peers is not allowed.
- You can consult any book, notes & Internet.
- Copy paste from Internet will give you **negative marks**.
- Lab work is divided into small tasks, complete all tasks sequentially.
- Show solution of each lab task to your Lab Instructor.
- In-Lab Exercises/Tasks
- **Write your code at provided space after each question.**
- **You need to upload code for all tasks at Google Classroom.**

# LAB TASKS

## Task 1 (20 mark)

Write a program that starts with a base class **employee**. This class contains employee's name and employee number. From this class inherit three other classes which are **manager**, **scientist** and **laborer**. The manager and scientist classes should contain additional information about these categories of employee and member functions.

Override the function which calculate salary of employee in its each child class.

**Manager class**:

- Additional attributes:
    - base_salary (float or integer) – Manager's base salary
    - bonus (float or integer) – Manager's bonus
- Additional member functions:
    - calculate_salary() – Overrides the base method to return salary including bonus
    - assign_task(task) – Assigns a task to the manager's department

**Scientist class**:

- Additional attributes:
    - base_salary (float or integer) – Scientist's base salary
    - research_allowance (float or integer) – Scientist's research allowance
- Additional member functions:
    - calculate_salary() – Overrides the base method to return salary including research allowance

- submit_research(title) – Submits a research report with the given title

**Laborer class**:

- Additional attributes:
  - hourly_rate (float or integer) – Laborer's hourly rate
  - hours_worked (integer) – Number of hours worked
- Member functions:
  - calculate_salary() – Calculates salary based on hourly rate and hours worked

## Program:

```
#include<iostream>

using namespace std;


class Employee {

public:

   string name;

   int employee_no;


   Employee(string n, int num) {

      name = n;

      employee_no = num;
```

```
    }

    void show_data() {
        cout << "Name: " << name << endl;
        cout << "Employee Number: " << employee_no << endl;
    }

protected:
    virtual int calculate_salary() = 0;
};

class Manager : public Employee {
public:
    int base_salary;
    int bonus;

    Manager(string n, int num, int base, int bon)
        : Employee(n, num), base_salary(base), bonus(bon) {}

    int get_salary() {
        return calculate_salary();
```

```
    }

protected:
    int calculate_salary() override {
        return base_salary + bonus;
    }
};

class Scientist : public Employee {
public:
    int base_salary;
    int research_allowance;

    Scientist(string n, int num, int base, int allowance)
        : Employee(n, num), base_salary(base), research_allowance(allowance) {}

    int get_salary() {
        return calculate_salary();
    }

protected:
```

```
    int calculate_salary() override {
        return base_salary + research_allowance;
    }
};

class Laborer : public Employee {
public:
    int hourly_rate;
    int hours_worked;

    Laborer(string n, int num, int rate, int hours)
        : Employee(n, num), hourly_rate(rate), hours_worked(hours) {}

    int get_salary() {
        return calculate_salary();
    }

protected:
    int calculate_salary() override {
        return hourly_rate * hours_worked;
    }
```

```
};

int main() {
    Manager m("Ali Raza", 1, 2000, 100);
    Scientist s("Furqan Ahmed", 2, 500, 7);
    Laborer l("Arslan Kiyani", 3, 10, 100);

    m.show_data();
    cout << "Manager Salary: " << m.get_salary() << endl;

    s.show_data();
    cout << "Scientist Salary: " << s.get_salary() << endl;

    l.show_data();
    cout << "Laborer Salary: " << l.get_salary() << endl;

    return 0;
}
```

**Output:**

```
Output

Name: Ali Raza
Employee Number: 1
Manager Salary: 2100
Name: Furqan Ahmed
Employee Number: 2
Scientist Salary: 507
Name: Arslan Kiyani
Employee Number: 3
Laborer Salary: 1000


=== Code Execution Successful ===
```

## Task 2 (20 mark)

See following real world objects;

- Medicine
- Tablet
- Syrup

Create a Python program using polymorphism with the following scenario:

- Define an abstract base class Medicine with attributes like name and manufacturer.
- Include an abstract method how_to_take() and a regular method display_info().
- Create two derived classes: Tablet and Syrup, each overriding how_to_take() with specific instructions.
- Demonstrate **polymorphism** by writing a function that takes a Medicine object and calls both display_info() and how_to_take() on it.
- Instantiate objects of both subclasses and show polymorphic behavior through iteration.

**Program:**

```
#include <iostream>
using namespace std;

class Medicine {
public:
    string name;
    string manufacturer;

    Medicine(string n, string m) {
        name = n;
        manufacturer = m;
    }

    void display_info() {
        cout << "Name: " << name <<endl;
        cout<<"Manufacturer: " << manufacturer << endl;
    }

    virtual void how_to_take() = 0;
};

class Tablet : public Medicine {
public:
    Tablet(string n, string m) : Medicine(n, m) {}

    void how_to_take() override {
        cout << "Take one tablet" << endl;
    }
};
```

```
class Syrup : public Medicine {
public:
    Syrup(string n, string m) : Medicine(n, m) {}

    void how_to_take() override {
        cout << "Take 10ml" << endl;
    }
};

int main() {
    Tablet tablet("Panadol", "islmabad  Pharmacy");
    Syrup syrup("arinac tablet ", "shifa labs");

    Medicine* meds[2] = { &tablet, &syrup };

    for (int i = 0; i < 2; ++i) {
        meds[i]->display_info();
        meds[i]->how_to_take();
    }

    return 0;
}
```

## Output:

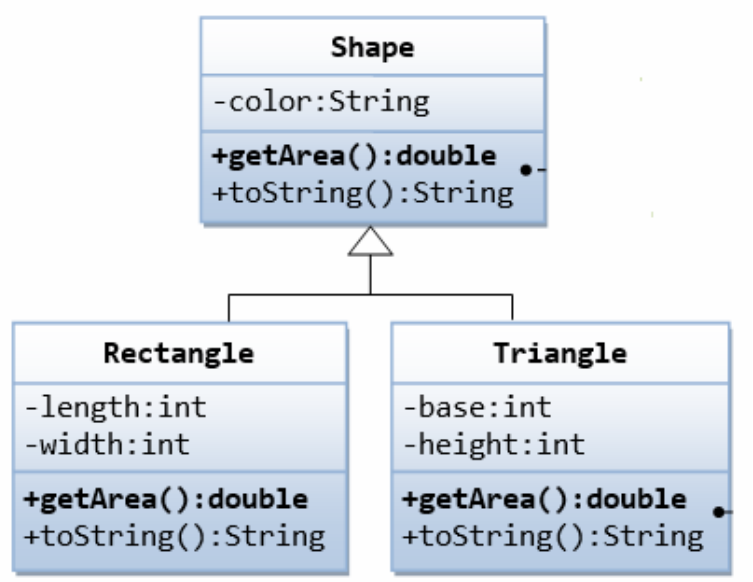
Output

```
Name: Panadol
Manufacturer: islmabad  Pharmacy
Take one tablet
Name: arinac tablet
Manufacturer: shifa labs
Take 10ml


=== Code Execution Successful ===
```

**Task 3** **(20 mark)**

See following UML diagram to complete the tasks.




You need to implement;

1. All Classes with their relationship
2. Implement **abstract** classes, **abstract** methods and **virtual** methods

**Program:**

```
#include <iostream>

using namespace std;



class Shape {

protected:

   string color;


public:
```

```
    Shape(string c) {
        color = c;
    }

    virtual double getArea() = 0;
    virtual string toString() {
        return "Shape";
    }
};

class Rectangle : public Shape {
private:
    int length;
    int width;

public:
    Rectangle(string c, int l, int w) : Shape(c) {
        length = l;
        width = w;
    }

    double getArea() override {
        return length * width;
    }

    string toString() override {
        return "Rectangle Color=" + color;
```

```
    }
};


class Triangle : public Shape {
private:
    int base;
    int height;

public:
    Triangle(string c, int b, int h) : Shape(c) {
        base = b;
        height = h;
    }

    double getArea() override {
        return 0.5 * base * height;
    }

    string toString() override {
        return "Triangle Color=" + color;
    }
};



int main() {
```

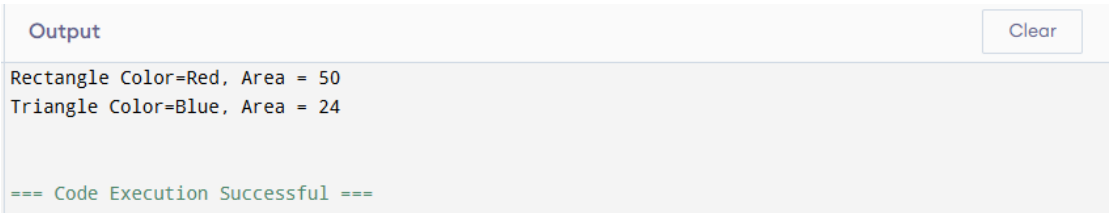
```
    Shape* r = new Rectangle("Red", 10, 5);
    Shape* t = new Triangle("Blue", 8, 6);

    cout << r->toString() << ", Area = " << r->getArea() << endl;
    cout << t->toString() << ", Area = " << t->getArea() << endl;

    delete r;
    delete t;

    return 0;
}
```

Output:




# EXTRA TASKS

Write a program to show calling of overridden function from the child classes defined in above tasks.